

IT5005: Artificial Intelligence

Assignment: Sudoku Knowledge Representation and Inference in Propositional Logic

1 What is Sudoku?

Sudoku is a constraint-satisfaction puzzle played on an $n \times n$ grid (conventionally $n = 9$, divided into 3×3 boxes). Some cells are pre-filled (*givens*); the task is to fill every remaining cell with a digit from 1 to n such that every row, every column, and every box contains each digit exactly once. A well-posed puzzle has exactly one solution consistent with its *givens*.

Sudoku has been studied extensively as a constraint-satisfaction and Boolean satisfiability problem:

- Simonis [1] formalizes Sudoku as a constraint problem and compares constraint-propagation strategies for solving it – directly relevant to the elimination-style reasoning this assignment asks you to encode as propositional rules.
- Lynce and Ouaknine [2] encode Sudoku directly as a Boolean satisfiability (SAT) instance and solve it with a SAT solver, the same underlying idea as this assignment’s general-clause representation, just with an industrial-strength solver in place of `pl_resolution/tt_entails`.
- McGuire, Tugemann, and Civario [3] settled a long-standing open question by proving, via exhaustive search, that no valid Sudoku puzzle has a unique solution with fewer than 17 givens.
- Crook [4] gives an accessible, purely pencil-and-paper account of the elimination and last-candidate techniques this assignment’s Horn-clause encoding is built from, without any propositional logic machinery; it’s useful as a sanity check on your encoding’s intent.
- The forward chaining, backward chaining, resolution, and model-checking algorithms themselves (and the `logic.py` implementations you are calling) follow Russell and Norvig [5], the standard reference for propositional inference procedures used throughout this course.

2 Problem Statement: Design the Sudoku Solver

The following subsections highlight the key steps required to solve the Sudoku puzzle.

2.1 Loading Puzzles from JSON

Sudoku puzzles are provided as JSON files (`puzzles.json`). The format of the JSON file for Sudoku puzzles is as follows:

```
{  
  "n": 9, "box_h": 3, "box_w": 3,
```

```

"puzzles": [
  {
    "givens": {"1_1": 3, "2_3": 1, ...},
    "given_count": 7,
    "solution": {"1_1": 3, "1_2": 4, ...}
  },
  ...
]
}

```

"r_c" string keys map to the value at row r , column c (1-indexed). JSON object keys must be strings, so this is the natural way to encode a (row, col) pair. The `given_count` is exactly how many cells are given: puzzles here are *not* labeled “easy/medium/hard,” since that scale assumes solving techniques (hidden pairs, X-Wing, backtracking) this assignment’s algorithms don’t use. The *given_count* is the number that actually determines whether a puzzle is solvable by pure elimination at all. The `solution` is included so you can check your own work, but your functions must not read `solution` to answer a query; they must only read `givens`. The starter notebook includes a loader function; you do not need to write your own JSON parsing.

2.2 Formalization of Sudoku Rules into Propositional Form (20 marks)

2.2.1 Define Symbols

Use two families of propositional symbols, for row r , column c , and value v (all ranging over 1 to n ; $n = 9$ for the puzzles in this assignment):

Symbol	Meaning
Is_{rcv}	cell (r, c) has value v
Not_{rcv}	cell (r, c) does <i>not</i> have value v

The helper function `atom(prefix, r, c, v)` for creating these symbols is provided for you in `sudoku_solver.py`; do not modify it.

2.2.2 Build KB

Every well-posed Sudoku puzzle satisfies exactly these conditions:

1. Every cell has *at least* one value from $\{1, \dots, n\}$.
2. Every cell has *at most* one value from $\{1, \dots, n\}$. (no cell holds two digits at once).
3. No two cells in the same row hold the same value.
4. No two cells in the same column hold the same value.
5. No two cells in the same box hold the same value.
6. The *givens* cells hold their stated values.

Formalize these rules as a propositional knowledge base (KB) in **two** representations: general clauses (no restrictions) and definite clauses. To this end, you need to implement two functions:

- `build_general_kb`, and

- `build_definite_kb`

Both functions take the puzzle's *givens* as fixed input facts. Implement `build_general_kb` and `build_definite_kb` in `sudoku_solver.py`. They must encode conditions 1–6 above and return `PropKB` and `PropDefiniteKB`, respectively.

2.3 Solve the Puzzle (30 marks)

Solving the puzzle involves finding the value entailed by the KB for a given cell. For example, if the puzzle contains an empty cell (r, c) and solving the puzzle by hand leads to a value v , then the query Is_{rcv} must yield a boolean value *True*.

(a) **Resolution and model checking on the general representation.** Using `build_general_kb` and the library's `pl_resolution` / `tt_entails` try to solve the puzzle, i.e., for each cell determine which value is entailed.

(b) **Forward chaining on the full grid.** Implement `solve_full_grid_fc` in `sudoku_solver.py`. Using your `build_definite_kb` and the library's `pl_fc_entails` (already implemented), solve the whole puzzle: for every cell, determine which value is entailed and reconstruct the solved grid.

(c) **Backward chaining.** Implement the following function in `sudoku_solver.py`:

```
pl_bc_entails(kb, query)  -> bool
```

Start from the query and recursively try to prove each premise of a rule whose conclusion matches the current goal, bottoming out at known facts. Your implementation should agree with `pl_fc_entails` on the cell/value queries used to verify the puzzle.

(d) **Backward chaining on the full grid.** Implement `solve_full_grid_bc` in `sudoku_solver.py`. Using your `build_definite_kb` and your own `pl_bc_entails`, solve the whole puzzle the same way as `solve_full_grid_fc`: for every cell, try each candidate value until `pl_bc_entails` confirms one. Time both `solve_full_grid_fc` and `solve_full_grid_bc` on the same puzzle and compare. Use your measured result where relevant in Question 5.

2.4 Answer the Queries (30 marks)

Answer each of the following questions directly in `Sudoku Assignment.ipynb`.

1. Detailed Representation Strategy: General vs. Definite (Horn) Encoding

Explain in detail how you formalized the Sudoku puzzle constraints into propositional logic across both Knowledge Base representations:

- General KB Strategy (`build_general_kb`):** Explain how you represented the standard Sudoku rules (e.g., at-least-one value per cell, at-most-one value per cell, row/column/box uniqueness) in propositional logic using the general KB representation, and how these constraints are represented as CNF clauses in the resulting `PropKB`.
- Definite KB Strategy (`build_definite_kb`):** Definite/Horn clauses strictly permit *at most one positive literal* per clause, prohibiting disjunctive constraints like $(Is_{r,c,1} \vee Is_{r,c,2} \vee \dots \vee Is_{r,c,n})$. Explain step-by-step how your encoding deals with this issue.

2. Theoretical Completeness vs. Computational Tractability

Model checking and resolution-refutation are sound and complete—they are guaranteed to terminate with a correct answer for any propositional KB. Despite this guarantee, explain

whether you would use either as the default algorithm for solving Sudoku puzzles. (*Hint: Consider space/time complexity and state-space growth.*)

3. Backward Chaining: Design, Pseudocode, and Challenges

Write pseudocode for `pl_bc_entails(kb, query)`, the backward-chaining algorithm you implemented in Section 2.3(c). Show, at a level of detail that reveals the algorithm's structure (not full Python), how the function checks whether the query is already a known fact, finds candidate rules whose conclusion matches the current goal, recursively proves each premise of such a rule, and combines results – both across the premises of one rule and across multiple candidate rules – to reach a single boolean answer. Then, in your own words, discuss the design challenges you had to work through to make your algorithm both correct and guaranteed to terminate on every puzzle, and explain how your pseudocode addresses them.

4. Expressive Limits of Horn Logic

Named elimination techniques such as Naked Pairs and X-Wing can, in fact, be encoded as definite clauses, using the same *Is/Not* vocabulary as your `build_definite_kb`. Work out how you would encode one of these techniques as definite clauses, and discuss the consequences of doing so.

5. Data-Driven vs. Goal-Driven Performance

Forward chaining (data-driven) and backward chaining (goal-driven) are both sound and complete for Horn KBs, but their execution runtimes vary depending on the target query.

- Describe a scenario—in terms of total KB size versus the query-relevant subset—where backward chaining is significantly faster than forward chaining.
- Describe a scenario where backward chaining offers no performance advantage, or performs worse than forward chaining.

Use your measured forward- vs. backward-chaining result where relevant.

3 Streamlit Integration (20 marks)

Build an interactive, user-friendly Streamlit application named `sudoku_app.py`. The app must import the core solver implementation from `sudoku_solver.py` rather than duplicating it. Your application must implement the following core requirements:

1. Puzzle Selection & Visual Board Display:

- Provide an interactive selector/dropdown to pick any puzzle from `puzzles.json`.
- Display the grid visually, clearly distinguishing initial *givens* from empty cells.

2. Full Grid Auto-Solver, with Algorithm Selection:

- Provide a control (e.g., radio buttons) allowing the user to choose forward chaining (`solve_full_grid_fc`) or backward chaining (`solve_full_grid_bc`).
- Provide a button that solves the full grid using the selected algorithm, renders the solved state, and displays the elapsed solve time.

3. Targeted Cell Entailment Query:

- Provide inputs for Row (*r*), Column (*c*), and Value (*v*).

- Include a button to execute a query checking whether Is_{rcv} is entailed using `pl.bc_entails`, displaying the boolean verdict (*True / False*).

4. User-Friendly Reasoning Trace:

- Select an appropriate inference algorithm (e.g., modified forward chaining or backward chaining) and instrument it to capture the dynamic reasoning steps taken during deduction.
- Present the reasoning trace in a visual, intuitive, and human-readable format rather than dumping raw Python strings or internal symbol dictionaries.
- *Recommended presentation strategies:*
 - Step-by-step visual board state animations or highlight sequences as numbers are deduced.
 - Expandable UI cards showing the rule-firing hierarchy (e.g., "*Inferred $Not_{1,2,3}$ because Row 1 already contains Value 3*" $\implies$ "*Deduce $Is_{1,2,4}$ as the last remaining candidate*").
 - Clear plain-English natural language explanations of eliminating options per row, column, or box constraint.

Import `atom`, `build_definite_kb`, `build_general_kb`, `solve_full_grid_fc`, `solve_full_grid_bc`, and `pl.bc_entails` from `sudoku_solver.py`. Do not copy or rewrite these core functions in `sudoku_app.py`. You may add app-specific helper functions where needed for the interface or reasoning trace.

Refer to the provided [StreamlitDeploymentGuide.pdf](#) to deploy your application. Paste the deployed Streamlit app URL in the designated section of `Sudoku.Assignment.ipynb`.

4 Deliverables

Submit only the following three files:

1. `Sudoku.Assignment.ipynb`, with all required cells run, outputs visible, all conceptual questions answered, and the deployed Streamlit app URL included in the designated section.
2. `sudoku_solver.py`, containing your implementation of the knowledge-base and inference functions.
3. `sudoku_app.py`, containing your Streamlit application.

The provided support files are required to run the assignment but are not part of the student submission. Rename the folder containing your three submission files with your group name and upload the zipped folder.

Note:

In `sudoku_solver.py`, import only from `utils.py` and `logic_.py`; do not edit either provided file. Do not duplicate the core solver functions in the notebook or `sudoku_app.py`.

References

- [1] H. Simonis. Sudoku as a Constraint Problem. *Proceedings of the CP Workshop on Modeling and Reformulating Constraint Satisfaction Problems*, 2005.
- [2] I. Lynce and J. Ouaknine. Sudoku as a SAT Problem. *Proceedings of the 9th International Symposium on Artificial Intelligence and Mathematics (AIMATH)*, 2006.
- [3] G. McGuire, B. Tugemann, and G. Civario. There Is No 16-Clue Sudoku: Solving the Sudoku Minimum Number of Clues Problem via Hitting Set Enumeration. *Experimental Mathematics*, 23(2):190–217, 2014.
- [4] J. F. Crook. A Pencil-and-Paper Algorithm for Solving Sudoku Puzzles. *Notices of the American Mathematical Society*, 56(4):460–468, 2009.
- [5] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.